

SECURITY AUDIT

Parallel

Fix

FINAL

REPORT



Disclaimer:

Security assessment projects are time-boxed and reliant on information provided by the client, its affiliates, or its partners. The findings in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase, and apply only to the specific commit hash, version, or deployment reviewed at the time of the audit. Any subsequent modification, redeployment, or upgrade is outside the scope of this report and may invalidate its findings.

The audit does not constitute investment, legal, financial, regulatory, or tax advice, nor an endorsement of any project or team. It is based on a limited review of materials provided and is delivered on an "as-is," "where-is," and "as-available" basis. Use of blockchain technology is subject to unknown risks and flaws. The scope of this assessment is technical and contract-level; unless explicitly stated otherwise, it does not cover economic or game-theoretic risks, MEV, oracle manipulation, governance attacks, centralization or admin-key risks, off-chain components, front-end code, or the behavior of third-party protocols, dependencies, libraries, or services interacted with by the audited code.

We make no warranties, express or implied, regarding the accuracy, reliability, completeness, or availability of the report or any associated services, and disclaim all implied warranties including merchantability, fitness for a particular purpose, and non-infringement. We assume no responsibility for any product, service, software, code, or material referenced by, linked to, or accessible through the report.

This report is prepared solely for the contract owner and may not be relied upon by any third party. The contract owner is responsible for making their own decisions based on the report and should seek additional professional advice as needed. To the maximum extent permitted by law, our total aggregate liability arising from or related to this audit shall not exceed the fees actually paid for the engagement. The contract owner agrees to indemnify and hold harmless the audit firm and its personnel from any claims, damages, expenses, or liabilities arising from use of or reliance on the report or the audited smart contract.

This disclaimer and any dispute arising from the audit shall be governed by the laws of Wyoming (USA), without regard to its conflict-of-laws principles. By engaging in this audit, the contract owner acknowledges and agrees to these terms.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Parallel – Fix – Audit Report
Website	app.parallel.best
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/parallel-protocol/parallel-parallelizer/commit/61942aacia0bca932c31b5c69d7ceb755cc97dc2
Resolution 1	https://github.com/parallel-protocol/parallel-parallelizer/commit/f0dde86f4ed39f88cb3569a2c9114283e47ea7a7

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium						
Low	4	4				
Informational	3	2		1		
Governance						
Total	7	6		1		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Savings

Savings is an upgradeable [ERC-4626](#) savings vault where deposited asset tokens earn protocol-controlled inflation over time through a governance-selected per-second rate. The vault issues [ERC-20](#) shares representing claims on the growing asset base, supports standard deposit/mint/withdraw/redeem flows, and adds [EIP-3009](#)-style authorized deposits and signed redemptions.

The core accounting invariant is based on `storedAssets`, not the raw [ERC-20](#) balance of the contract. `storedAssets` is only updated through deposits, mints, withdrawals, redemptions, authorized flows, and `_accrue`. Direct token transfers to the contract are ignored by `totalAssets()`, preventing donation/inflation attacks against share pricing. Any excess balance above `storedAssets` is treated as surplus and can be recovered by governance through `recoverSurplus`.

Yield accrues lazily on user interactions or governance rate changes. `_accrue()` computes the updated asset balance using a third-order approximation of compounded growth, mints the earned underlying asset to the vault via `ITokenP[asset()].mint`, updates `storedAssets`, and emits `Accrued`. `totalAssets()` returns the projected accrued value of `storedAssets` since `lastUpdate`.

The contract can be paused by governance. Pausing first accrues yield up to the pause timestamp, then blocks deposits, mints, withdrawals, redemptions, and authorized variants. Unpausing advances `lastUpdate` to the current timestamp, so no yield accrues during the paused interval.

Authorized flows allow gasless or relayed interactions. `depositWithAuthorization` consumes a Savings-specific EIP-712 authorization binding the receiver, then pulls assets from the owner using the underlying token's EIP-3009 `receiveWithAuthorization`. `redeemWithAuthorization` consumes a signed authorization over the shares and sends the redeemed assets to the bound receiver, supporting both EOA signatures and EIP-1271-compatible contract wallets.

Privileged Functions

- initializeStoredAssets
- pause
- unpause
- toggleTrusted
- setRate
- setMaxRate
- recoverSurplus

Invariants

- INV-1: `totalAssets()` must be based on projected `storedAssets`, not `asset().balanceOf(address(this))`, so direct donations cannot inflate share value.
- INV-2: `storedAssets` must increase on every legitimate deposit/mint/authorized deposit and decrease on every withdrawal/redeem/authorized redeem.
- INV-3: `_accrue()` must settle yield using the previous rate and `lastUpdate` before mutating rate-related state.
- INV-4: rate must never exceed `maxRate` after `setRate` or `setMaxRate`.
- INV-5: When paused, all user-facing vault interactions must be blocked: deposit, mint, withdraw, redeem, `depositWithAuthorization`, and `redeemWithAuthorization`.
- INV-6: Paused time must not generate yield: `pause()` accrues up to the pause moment, and `unpause()` resets `lastUpdate` to the unpause timestamp.
- INV-7: Directly transferred surplus assets must only be recoverable when `actualBalance > storedAssets`, and recovery must never reduce assets backing shareholders.
- INV-8: Signed deposit and redeem authorizations must bind the intended receiver, preventing relayers from redirecting minted shares or redeemed assets.

Issue_01	<code>initializeStoredAssets</code> selector is never granted a role, so the prescribed atomic upgrade reverts in production
Severity	Low
Description	The documented upgrade procedure runs the seeding atomically as <code>upgradeToAndCall</code> with <code>initializeStoredAssets</code> encoded as the call data. The function carries the restricted modifier. When it executes nested inside <code>upgradeToAndCall</code> the modifier resolves the caller's permission against the <code>initializeStoredAssets</code> selector through <code>_msgData</code>, not against <code>upgradeToAndCall</code>. The savings role script grants <code>upgradeToAndCall</code>, <code>setMaxRate</code>, <code>recoverSurplus</code> and others to the governor but never grants <code>initializeStoredAssets</code> to any role. In OpenZeppelin <code>AccessManager</code> an unconfigured selector defaults to <code>ADMIN_ROLE</code>. So the nested call only passes if the upgrading governor also holds the <code>AccessManager</code> admin role. This is masked by the test suite. The test fixture deploys the <code>AccessManager</code> with the governor as initial admin, so in tests the governor satisfies the <code>ADMIN_ROLE</code> default and the atomic upgrade succeeds. Production deployments may separate the governor from the <code>AccessManager</code> admin, in which case the nested call reverts with <code>AccessManagedUnauthorized</code> and the entire upgrade transaction fails.
Recommendations	Grant the <code>initializeStoredAssets</code> selector to the role that performs the upgrade before executing it.
Comments / Resolution	Fixed by granting the role in the setup script.

Issue_02	<code>initializeStoredAssets</code> does not re-anchor <code>lastUpdate</code>
Severity	Low
Description	<code>initializeStoredAssets</code> seeds <code>storedAssets</code> , but does not set <code>lastUpdate</code> to the current time. The first accrual after the upgrade, therefore, mints yield over the pre-upgrade idle window on the freshly seeded base. The effect is bounded by the thirty-minute staleness guard and is consistent with how the legacy contract projected over the same window. A prior report fixed the same anchoring class in the constructor path, but not in this reinitializer.
Recommendations	Set <code>lastUpdate</code> to the current time inside <code>initializeStoredAssets</code> if the upgrade is intended to be an accrual checkpoint, or document that the idle window is intentionally minted on the seeded base.
Comments / Resolution	Fixed following recommendations.

Issue_03	<code>totalAssets</code> accrues yield while <code>Savings</code> is paused
Severity	Low
Description	The <code>Savings</code> contract is designed to halt emissions while paused. When <code>pause</code> is called, the contract settles all yield up to the pause timestamp and sets <code>paused</code> to 1. When <code>unpause</code> is called, <code>lastUpdate</code> is moved to the unpause timestamp so the paused interval is not later minted as yield. However, the ERC4626 <code>totalAssets</code> view does not account for the paused state: <pre>function totalAssets() public view override returns (uint256) { return _computeUpdatedAssets(storedAssets, block.timestamp - lastUpdate); }</pre> After <code>pause</code> settles yield, <code>lastUpdate</code> is equal to the pause timestamp. As time passes while paused, <code>totalAssets</code> keeps calling <code>_computeUpdatedAssets</code> with the elapsed paused duration, so it reports yield that should not accrue during the pause. This creates inconsistent view accounting during the paused period. Deposits, mints, withdrawals, and redemptions are blocked, but external integrations, accounting systems, or frontends that rely on <code>totalAssets</code> can temporarily overvalue <code>Savings</code> shares.
Recommendations	Make <code>totalAssets</code> return <code>storedAssets</code> while the contract is paused, and only project future yield while the contract is active.
Comments / Resolution	Fixed.

Issue_05	setRate and setMaxRate mint paused yield
Severity	Low
Description	The Savings contract is designed so pausing halts yield emission. pause settles all yield accrued up to the pause timestamp, while unpaue advances lastUpdate to the unpaue timestamp so the paused interval is discarded. However, setRate and setMaxRate are still callable while the contract is paused, and both functions call _accrue before updating their respective parameters. Since lastUpdate is set to the pause timestamp, any later call to setRate or setMaxRate during the pause makes _accrue mint yield for the paused duration. As a result, an emergency pause does not fully stop Savings emissions if a trusted updater or governance changes rate parameters while paused. The contract can mint tokenP to itself and increase assets per share during a period where yield was expected to be halted. The amount depends on the active rate, stored asset balance, and paused duration before the setter is called.
Recommendations	Do not accrue yield from rate setters while the contract is paused.
Comments / Resolution	Fixed.

Issue_06	Overridden ERC4626 actions bypass max limit checks
Severity	Informational
Description	Savings overrides the ERC4626 max functions so that the vault reports zero capacity while paused. However, the overridden <code>deposit</code>, <code>mint</code>, <code>withdraw</code>, and <code>redeem</code> functions do not call the parent ERC4626 implementation and do not reproduce its max checks. For example, <code>deposit</code> accrues yield, computes shares, and directly calls <code>_deposit</code>: <pre> function deposit(uint256 assets, address receiver) public override whenNotPaused returns (uint256 shares) { uint256 newTotalAssets = _accrue(); shares = _convertToShares(assets, newTotalAssets, Math.Rounding.Floor); _deposit(_msgSender(), receiver, assets, shares); } </pre> In the current implementation, the pause limit is independently enforced by the <code>whenNotPaused</code> modifier, which means the issue does not currently allow users to act while paused. The remaining impact is that the contract is not ERC4626-limit compliant, and any current or future limit expressed through the max functions, such as deposit caps, receiver restrictions, liquidity throttles, or stricter withdrawal limits, can be bypassed if not called by the overridden functions.
Recommendations	Preserve the parent ERC4626 max checks in each overridden action, namely <code>deposit</code> , <code>mint</code> , <code>withdraw</code> , and <code>redeem</code> .
Comments / Resolution	Fixed.

Issue_07	<code>initializeStoredAssets</code> can run once on a fresh deployment
Severity	Informational
Description	The original initializer sets the contract to version one, and <code>initializeStoredAssets</code> is a version two reinitializer, so on a fresh deployment, it remains callable once. On a fresh vault <code>storedAssets</code> is already seeded correctly by the initialization deposit, so re-running it just overwrites <code>storedAssets</code> with the current balance, which is harmless unless a donation arrived first. The call is restricted, so the practical risk is minimal.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_08	Non-atomic upgrade leaves <code>storedAssets</code> at zero, breaking deposits and letting redeemers burn shares for nothing
Severity	Informational
Description	The upgrade makes <code>storedAssets</code> the single source of truth for backing, and <code>totalAssets</code> now returns the projection of <code>storedAssets</code> rather than the raw token balance. That <code>storedAssets</code> value is seeded only by <code>initializeStoredAssets</code>, a one-shot reinitializer that is meant to run during the upgrade. If the upgrade is executed without bundling that call atomically, the contract goes live with <code>storedAssets</code> equal to zero while <code>totalSupply</code> is still the full pre-upgrade share supply. In that state <code>totalAssets</code> returns zero and the vault is still open, gated only by <code>whenNotPaused</code>. deposit and withdraw revert because the share conversion divides by a zero <code>totalAssets</code>, which reverts inside the OpenZeppelin <code>mulDiv</code>. redeem is worse: the asset conversion computes shares times zero divided by supply, returns zero, and the contract proceeds to burn the holder's shares while transferring zero underlying. The <code>storedAssets</code> decrement is by zero so nothing underflows and the call succeeds, leaving the redeemer with no shares and no assets. The <code>storedAssets</code> equal to zero state is not reachable through normal operation. The seed deposit performed at initialization mints dead shares to the contract and keeps <code>storedAssets</code> at or above the seed amount for the life of the vault. It only arises if the upgrade is performed as two separate transactions, or if the atomic call reverts for some other reason and an operator pushes the implementation live anyway. The access control and staleness issues described below can both force exactly that non-atomic path.
Recommendations	Enforce atomicity in code rather than by deployment convention. Guard the ERC4626 entry points so they revert while uninitialized, for example by requiring <code>storedAssets</code> to be non-zero whenever <code>totalSupply</code> is non-zero, so a half-finished upgrade cannot mint or burn shares against a zero backing.
Comments / Resolution	Fixed by introducing the <code>onlyInitialized</code> modifier.